

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: Student Performance Analysis and Management System

Team Members:

- Team Member 1 – Frontend Developer
- Team Member 2 – Backend Developer
- Team Member 3 – Database Developer
- Team Member 4 – Testing and Documentation

The project is developed using the **MERN Stack**, which consists of MongoDB, Express.js, React.js, and Node.js. The application provides an easy way to manage student information, marks, and performance details through a web-based system.

2. Project Overview

Purpose

The purpose of this project is to develop a web application for managing and analyzing student performance. It helps users store student details, manage marks, and view performance information in an organized way.

The system reduces manual work and provides a simple interface for accessing student-related information.

Features

The main features of the application are:

- Student registration and management.

- Adding and updating student details.
- Adding and managing subject marks.
- Viewing student performance.
- Searching student records.
- Displaying performance information in an organized format.
- User authentication and authorization.
- Responsive and user-friendly interface.
- Secure communication between frontend and backend.

3. Architecture

The MERN application consists of three main layers: **Frontend, Backend, and Database.**

Frontend – React.js

React.js is used to develop the frontend of the application. It provides reusable components and helps create an interactive user interface.

The frontend communicates with the backend using API requests. React components are used for pages such as login, dashboard, student details, marks, and performance reports.

Backend – Node.js and Express.js

Node.js is used to run the server-side application. Express.js is used as the backend framework to create APIs and handle client requests.

The backend receives requests from the React frontend, processes the required information, communicates with MongoDB, and sends the response back to the frontend.

Database – MongoDB

MongoDB is used to store application data. It is a NoSQL database that stores information in document format.

The database can contain collections such as:

- Users
- Students

- Subjects
- Marks
- Performance

The backend uses MongoDB to insert, retrieve, update, and delete data.

4. Setup Instructions

Prerequisites

The following software is required to run the project:

- Node.js
- npm
- MongoDB
- Visual Studio Code or another code editor
- Web browser
- Git/GitHub

Installation

1. Download or clone the project from the repository.
2. Open the project folder in Visual Studio Code.
3. Open the terminal.
4. Navigate to the client folder.
5. Install the required frontend packages using `npm install`.
6. Navigate to the server folder.
7. Install the required backend packages using `npm install`.
8. Create the required environment variables.
9. Add the MongoDB connection string to the environment file.
10. Start the frontend and backend servers.

5. Folder Structure

Client

The **Client** folder contains the React frontend application.

Example structure:

```
client/
|
├─ public/
├─ src/
|   ├─ components/
|   ├─ pages/
|   ├─ services/
|   ├─ App.js
|   └─ index.js
├─ package.json
└─ README.md
```

- **components:** Contains reusable React components.
- **pages:** Contains different application pages.
- **services:** Handles API communication.
- **App.js:** Main React application component.
- **index.js:** Entry point of the React application.

Server

The **Server** folder contains the Node.js and Express.js backend.

```
server/
|
├─ controllers/
├─ models/
├─ routes/
├─ middleware/
├─ config/
├─ server.js
├─ .env
└─ package.json
```

- **controllers:** Contains application logic.
- **models:** Defines MongoDB data models.
- **routes:** Defines API endpoints.
- **middleware:** Handles authentication and other common functions.
- **config:** Contains configuration settings.
- **server.js:** Starts the backend server.

6. Running the Application

The frontend and backend are started separately.

Frontend

Open the terminal and move to the client directory:

```
cd client
npm install
npm start
```

The React application will start on the local development server.

Backend

Open another terminal and move to the server directory:

```
cd server
npm install
npm start
```

The Node.js and Express.js backend server will start locally.

7. API Documentation

The backend provides APIs for communication between the frontend and database.

Method	Endpoint	Purpose
POST	/api/auth/login	User login

POST	/api/auth/register	Register a new user
GET	/api/students	Get all students
GET	/api/students/:id	Get a particular student
POST	/api/students	Add a new student
PUT	/api/students/:id	Update student details
DELETE	/api/students/:id	Delete a student
GET	/api/marks	Get marks
POST	/api/marks	Add marks

Example Request

POST /api/students

Example data:

```
{
  "name": "Arun",
  "rollNumber": "101",
  "department": "Computer Science"
}
```

Example Response

```
{
  "message": "Student added successfully",
  "student": {
    "name": "Arun",
    "rollNumber": "101",
    "department": "Computer Science"
  }
}
```

8. Authentication

Authentication is used to protect the application from unauthorized access.

Users first provide their login credentials. The backend verifies the credentials and provides an authentication token when the login is successful.

The token is then used to access protected resources. Authorization ensures that only permitted users can perform specific operations such as adding, editing, or deleting student records.

Passwords should be stored securely using hashing rather than storing them as plain text.

9. User Interface

The application provides a simple and user-friendly interface.

The major UI screens include:

- Login Page
- Registration Page
- Home Page
- Dashboard
- Student Management Page
- Marks Entry Page
- Student Performance Page
- Profile Page

Screenshots or GIFs can be added to this section to demonstrate the working of each feature.

[Insert Screenshot – Login Page]

[Insert Screenshot – Dashboard]

[Insert Screenshot – Student Details]

[Insert Screenshot – Performance Page]

10. Testing

Testing is performed to verify that the application works correctly and meets the project requirements.

Different types of testing can be performed:

- **Unit Testing:** Tests individual components or functions.
- **Integration Testing:** Tests communication between frontend, backend, and database.
- **API Testing:** Verifies that API endpoints work correctly.
- **Functional Testing:** Checks whether each feature performs its required function.
- **User Acceptance Testing:** Ensures the application satisfies user requirements.

Tools such as Postman, browser developer tools, and testing frameworks can be used for testing.

11. Screenshots or Demo

Screenshots of the working application are included to demonstrate the major features.

The screenshots may include:

1. Login screen
2. Registration screen
3. Dashboard
4. Student list
5. Add student page
6. Marks entry page
7. Performance analysis page
8. Database/API output

A demo link can also be provided if the application is deployed online.

Demo Link: [Add project demo link here]

12. Known Issues

The following issues may be identified during development or testing:

- Internet connection may be required for some services.
- Incorrect login details will prevent access to protected pages.
- The application may require proper MongoDB configuration.
- Some features may depend on the backend server being active.
- Validation may be required for certain input fields.
- Performance may decrease when handling a very large amount of data.

These issues can be fixed or improved in future versions.

13. Future Enhancements

The project can be improved by adding more features in the future.

- Add advanced student performance charts and graphs.
- Add email notifications.
- Add PDF report generation.
- Add attendance management.
- Add teacher and administrator dashboards.
- Add advanced search and filtering.
- Improve security and authentication.
- Deploy the application to a cloud platform.
- Add mobile-friendly design.
- Add automated testing.
- Add data backup and recovery features.
- Add AI-based student performance prediction.

These enhancements can make the application more useful, secure, and scalable.